Configuration-Driven Integration of Industrial Agents Across Production Scenes

***Abstract*—Industrial agents can propose setpoint changes, but executing a proposal requires preserving its production context, signal meaning, write authority, and process limits. We present a configuration-driven runtime that binds production context and signal semantics to named operations and registered protocol drivers. Registered Agent harness adapters and direct clients submit operations to a shared dispatcher. Write handlers resolve live context at execution and apply authorization and configured value limits. Routes that require approval hold the request pending and repeat their route-specific checks after approval. Command receipts, actuator readbacks, and later process observations remain separate records. Probes across four simulator scenes exercised selected node-to-driver mappings: 220 admissible writes were accepted, 219 collected structural-range requests were rejected, and 216 available readbacks matched. In deterministic missions across three scenes, a scripted policy reached the target band in all 15 seed–scene runs; no-write conditions reached it in none. This contrast is descriptive because the conditions differed in exposure time and mission logic. With the process-window interlock enabled, none of six targeted violations reached the mock driver. Removing the interlock sent all six to the driver while structural checks remained active. These results provide evidence for the tested integration and interlock paths in simulation; they do not establish live-Agent performance or physical-plant safety.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, system integration.**

I. Introduction

INDUSTRIAL agents can propose parameter changes, but a proposal becomes an industrial operation only after the system resolves its production context and signal meaning. Each write must be tied to the active line, product, recipe, and run; mapped to a configured control point; and checked against current authority and process limits. The process measurement used to assess a response is distinct from the command signal. In the cast-film scene studied here, line speed is the writable setpoint and film thickness is measured separately, although both use Message Queuing Telemetry Transport (MQTT). A command receipt or setpoint readback therefore does not establish the film response. Fig. 1 illustrates the distinction.

Research on industrial agents addresses service supervision, control generation, programmable logic controller (PLC) programming, and multi-agent manufacturing workflows [1]–[10]. Other work exposes manufacturing capabilities through standardized tools and semantic skills [11]–[13], or combines agent proposals with asset models, approval, audit, and process verification [14]–[16]. Together, these studies address agent organization, plant-facing tools, and operation governance. The design problem is to carry the active line–product–recipe–run context and signal meaning through authorization and protocol dispatch, then return evidence that separates the command outcome from the process response.

We implement this path as a configuration-driven runtime. A production-scene description binds the line–product–recipe–run context to semantic acquisition and control nodes, protocol drivers, process objectives, and write limits. Task-mediated and direct operation calls enter a shared dispatcher, where route handlers resolve live state again before device access. Service-owned records keep the requested command, optional device readback, and later process observation distinct. Through a registered harness, the Agent decides whether to request another task-level step; the runtime applies the configured checks to each operation. The contribution is an implemented system-integration architecture, not a new control law or agent-reasoning algorithm.

The contributions are:

1) A production-scene configuration model that binds operational context to semantic signal acquisition and control, supported drivers, process objectives, and engineering limits behind named operations.
2) A task-to-device path that exposes the same named operations through registered Agent harness adapters and direct callers. Both paths use shared dispatch checks; route-specific handlers apply current admission and approval rules and retain command evidence separately from process observations.
3) An evaluation of selected scene–protocol mappings, admissible writes, structural rejections, and available readbacks across four simulator scenes, followed by a five-seed deterministic-mission comparison and a process-window interlock ablation. Each block uses its own unit and denominator, so simulator outcomes remain separate from mock-driver results.

Section II reviews industrial-agent and plant-integration work. Section III presents the architecture and scene configuration. Section IV follows one task-level operation through its feedback path, and Section V details write governance and evidence. Section VI reports the simulator evaluation. Section VII discusses what these results support and what remains untested.

II. Related Work

Work relevant to this system spans industrial-agent supervision, agent-facing manufacturing interfaces, and operation governance. The systems differ in their goals and evaluation methods. Table I summarizes their mechanisms and reported evidence without ranking their results.

A. Industrial Agents and Plant Integration

Industrial agents have been used to supervise production services, generate control actions, program programmable

logic controllers (PLCs), and coordinate manufacturing tasks. Biskupovic et al. study online selection and reselection of services in service-oriented industrial cyber–physical systems [1]. Xia et al. describe an architecture connecting language-model agents, digital twins, and automation systems [17]; other work examines agent-generated control and PLC programs [2], [5], [6]. Recent multi-agent work also evaluates module-level agents and skill execution in simulated manufacturing settings [4], [8]. These studies focus primarily on agent organization, control generation, or production coordination.

Agent-facing interfaces provide a second point of comparison. Vieira da Silva et al. examine MCP for exposing manufacturing capabilities as tools, while Köhle et al. compose semantic recipes from OPC Unified Architecture (OPC UA) skills [11], [12]. Xavier et al. present IndustriConnect, a recent arXiv preprint on MCP adapters for Modbus, MQTT/Sparkplug B, and OPC UA, evaluated with a mock-first workflow and deterministic protocol tests [13]. A closely related study at the 2026 IEEE International Conference on Automation Science and Engineering (CASE) uses agents and MCP to commission a cognitive digital twin by extracting specifications and binding live telemetry; it reports robotic-cell validation and commissioning-time results [10]. Its focus is digital-twin commissioning, whereas the runtime studied here governs named operations after scene configuration and separates command evidence from process observations. The 2026 IEEE International Conference on Emerging Technologies and Factory Automation (ETFA) paper by Köhle et al. combines module-specific agents, MCP-wrapped OPC UA skills, MQTT coordination, and injected factory state, and reports nine simulated production challenges [4]. These results make tool interoperability and simulated manufacturing execution close comparators. IndustriConnect focuses on protocol adapters, whereas the ETFA study evaluates agent coordination and module skills. Here, we focus on binding a configured production tuple to semantic signal nodes and a shared operation path with route-specific governance. The experiments assess these integration paths; they do not compare system performance.

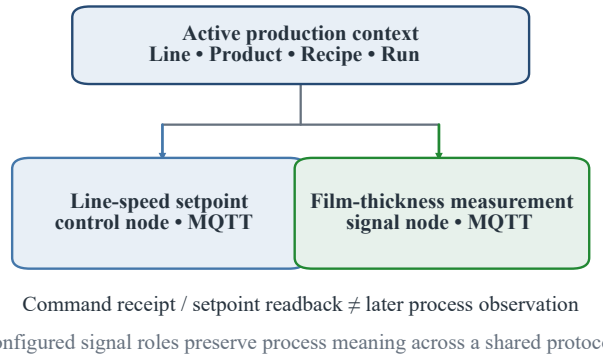


Fig. 1. In the cast-film scene, the line-speed setpoint and film-thickness measurement are separate configured nodes even though both use MQTT. A command receipt or optional setpoint readback is distinct from the later process observation.

B. Governed Agent Operations

Safety and audit mechanisms have been integrated with agent execution in several forms. Graul et al. ground proposals in the Asset Administration Shell (AAS) and use signed approval, PLC-validated OPC UA Methods, and audit logging [14]. Schall separates large language model (LLM) proposals from verification grounded in piping and instrumentation diagrams (P&IDs), supervisory control and data acquisition (SCADA), and standard operating procedures (SOPs), while Lin and Chen report a policy-governed sensor–manufacturing execution system (MES)–enterprise resource planning (ERP) digital thread [15], [16]. The runtime studied here applies admission and approval between a named operation and its configured driver. PLCs, safety instrumented systems (SIS), and other plant-protection functions remain separate safeguards.

Table I compares the operation boundary and evaluation type reported in representative work. Published simulation and laboratory results are not directly comparable because the systems, tasks, and denominators differ.

C. Industrial Context and Operation Semantics

MCP standardizes tool discovery and invocation, while operation-level scope, policy, grant, and resource-binding checks remain application concerns [18]. ISA-88 and ISA-95 define recipe and production-operation concepts [19], [20]; run-to-run control relates process observations to later production decisions [21]. Human–automation research analyzes how information acquisition, decision, and action responsibilities are allocated [22]. These standards and studies define the production entities and human–automation responsibilities an industrial runtime must preserve. Section III shows how the system carries them into configured operations; the evaluation tests selected paths in simulation.

III. System Design

The runtime has two entry paths to a shared set of named services. Task-mediated calls pass through the collaboration runtime and a registered Agent harness. Direct calls through the Representational State Transfer (REST) API or the standard input/output (stdio) transport for the Model Context Protocol (MCP) enter the shared dispatcher without creating a task turn. Both paths use the same scene configuration for production context, semantic-node mappings, protocol endpoints, objectives, and write limits.

A. Design Rationale and Requirements

The runtime must preserve process meaning across heterogeneous drivers, provide a stable operation contract for registered harness adapters, and check current authority and process limits before each write. It must also distinguish command acceptance from later process observations. Scene configuration, registered adapters, shared dispatch, route handlers, and service-owned records enforce these boundaries. Existing named operations can use any registered driver that satisfies their interface; a new protocol still requires a driver

TABLE I
REPRESENTATIVE PLANT-FACING AGENT SYSTEMS AND THE SCOPE OF THEIR REPORTED EVALUATIONS.

System	Reported operation or integration mechanism	Reported evaluation
Biskupovic et al. [1]	Agent-based service selection and online reselection using performance and computational-load metrics	Industrial cyber–physical system study; service adaptability and performance
Xia et al. [17]	Cognitive, digital-twin, and physical-automation layers with twin snapshots, events, and plans	Robotic automation and process-simulation cases
Köhle et al. [4]	Module-specific agents, MCP-to-OPC-UA skills, MQTT coordination, and injected factory state	Nine simulated production challenges; accepted ETFA 2026 paper
Xavier et al. [13]	MCP adapters for industrial protocols with a common response envelope and mock-first testing	arXiv preprint: 870 benchmark runs and 2,820 tool calls
Liu et al. [10]	Agentic semantic commissioning of a cognitive digital twin with MCP-based telemetry binding	Robotic machining cell; reported 97.2% perception mean average precision (mAP) and commissioning in about 2 h
Graul et al. [14]	AAS-grounded proposals, signed approval, PLC-validated OPC UA Methods, and audit log	Two laboratory proof-of-concept settings
Schall [15]	P&ID-, SCADA-, and SOP-grounded proposal verification with gated execution	Five simulated scenarios, 43 proposals, and 50 LLM runs
Lin and Chen [16]	Policy-governed sensor–MES–ERP digital thread	Manufacturing-record evaluation with operational metrics
This work	Scene-bound production context and signal mappings; shared dispatch with route-specific write governance and separate command receipts and process-observation evidence	Two integrated pipeline executions (77 assertions each), a four-scene probe subset, five-seed scripted missions, and a three-repeat mock-driver process-window ablation

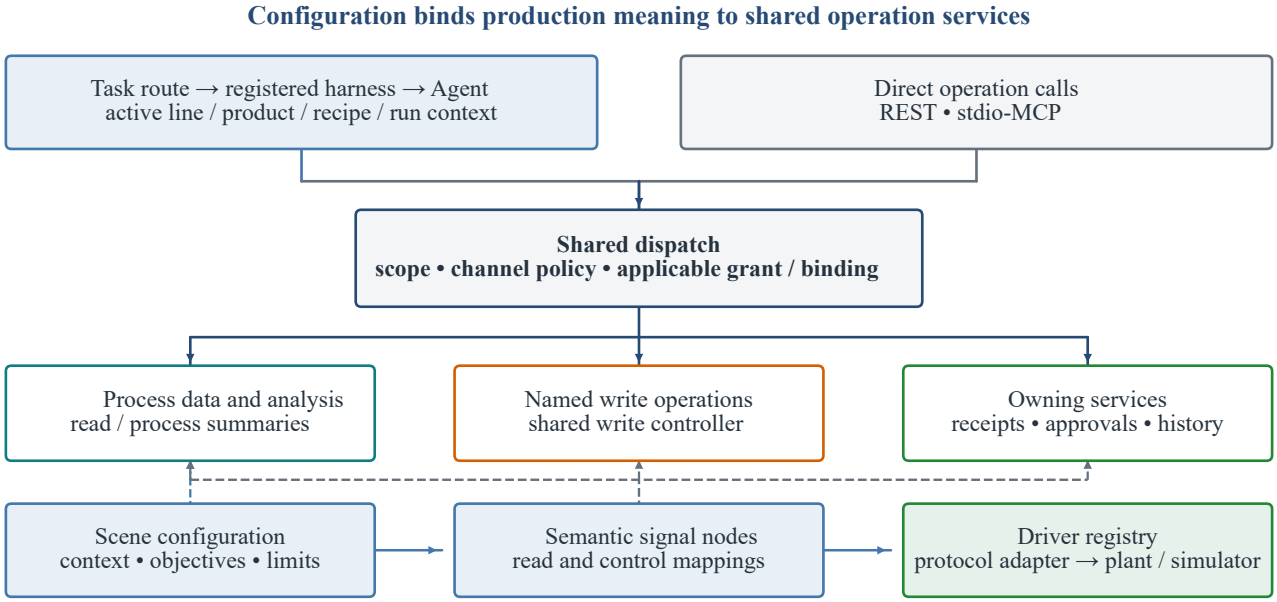


Fig. 2. The system routes task-mediated and direct operation calls through shared dispatch and service handlers. Solid arrows show operation flow and configured field mappings; dashed connectors associate the scene configuration with the services that consume it. Named write operations use the shared write controller, while service-owned records keep approvals, command receipts, optional readbacks, and later observations distinct.

adapter. Section VI evaluates selected configured simulator paths.

At task start, the context builder assembles the active production tuple and relevant process summaries. This context informs a proposal but does not confer write authority. Every write resolves live state again, including direct calls at invocation.

B. Layered Architecture

Fig. 2 shows the system layers and their interfaces. Task and chat routes use the collaboration runtime, which provides workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. An adapter connects an agent engine to its task interface; its binding and channel policy constrain the tools available to that agent.

Direct REST calls and the stdio-MCP operation bridge bypass task execution and enter the shared dispatcher. They create neither a task nor a harness turn; MCP management and task methods remain on their service routes. The scene configuration has two consumers: the task route receives active context and objectives, while operation services resolve node mappings and guards when an operation executes. The service band provides data acquisition (DAQ), write control, analysis, retrieval, and optional model services. At the field layer,

TABLE II
SCENE CONFIGURATION ELEMENTS AND THEIR ROLES IN THE RUNTIME PATH.

Element	Configured information	Runtime use
Production context	Line, product, recipe, and run associations	Context construction for a task turn; live context resolution before a write
Semantic I/O	Process variable, direction, unit, and node mapping	Acquisition uses the read mapping; accepted operations resolve the configured control mapping
Driver binding	Supported protocol driver and endpoint configuration	Driver registry selects the protocol implementation after operation checks
Operation admission	Engineering, baseline, product, and recipe limits; step and timing rules where configured	Route handler checks the candidate against current bounds and rejects inadmissible requests
Approval policy	Whether approval is required and the decision validity window	Pending request binds the proposed action; route-specific checks run after approval
Evidence ownership	Command receipt, verification grade, and later DAQ observations	Owning services retain command outcomes separately from process measurements

input/output (I/O) mappings connect semantic nodes to built-in or plugin drivers for a plant or simulator. Each owning service retains its task, command, observation, or configuration records and associates them with the actor and production context where supported.

C. Scene Configuration as the Integration Boundary

Scene-specific process semantics are represented in configuration before an agent task begins. The active line identifies the production context; product and recipe records supply applicable targets and limits; the run record provides the current operating state. Semantic nodes bind process variables to direction-specific acquisition or control mappings, and the driver registry resolves each mapping to a supported protocol implementation. The agent-facing operation remains named and typed, so a harness does not need to construct protocol-specific messages. A scene using an already registered driver can supply different context and node mappings behind the same operation contract. A protocol not present in the registry still requires a driver adapter; configuration alone does not implement a new protocol.

Table II summarizes how the scene description is consumed along the execution path. It distinguishes configuration-time bindings from checks against live state. The context assembled for an agent turn is descriptive: before a write, the handler resolves the active context again and applies the current limits. Approval is bound to the proposal and to the context where that route supports context snapshots; the command receipt remains separate from subsequent DAQ observations.

D. Cross-Module Execution Flow

For a task-mediated call, the registered adapter presents a scoped tool catalog and current task context to the Agent. At the operation boundary, the shared dispatcher checks the predicates declared for the named operation before selecting its handler. The handler resolves live production state, applies route-specific guards and limits, and requests any required approval before device I/O.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a, c, u) = \bigwedge_{X \in \mathcal{C}(u)} X(a, c, u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, applicable industrial grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S, P, G, B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is included when the resolved channel, target line, and owner/grant policy require it; explicit exemptions omit it. The binding predicate B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. After these predicates pass, the selected route applies its guards, active-run checks, admission limits, and any required approval. If approval is required, the handler applies the post-approval checks defined for that route before device dispatch. Recipe operations revalidate the active context; the single-node path rechecks binding, channel grant, and current write guards.

For a direct REST call, the service establishes a and c before applying these checks. An agent token must match the requested agent identifier; an authenticated user must be the channel owner or an administrator. The manager loads the channel-agent record and rechecks scope and policy, followed by any applicable grant and binding checks. Handlers resolve live production state from request identifiers and owning services.

Section V specifies route-level admission, timing, approval, and evidence rules for both task-mediated and direct calls.

IV. AGENT-COORDINATED TASK-LEVEL OPERATION

The runtime exposes a task-level operation, not an autonomous continuous controller. In each task turn, the Agent receives the active production context and chooses whether to inspect, propose one parameter step, or stop. A proposal calls the named `optimization_explore` operation. The operation validates the active optimization profile and control-node binding, then reads the stored control-node value as the current setpoint u_0. This stored value is not necessarily an independent device readback.

Given a direction and step Δu, the operation forms a candidate $u^* = \mathrm{clip}(u_0 \pm \Delta u)$ within the configured tuning and node bounds. It then resolves the target DAQ binding and checks its visibility before calling the shared write-control path; an initial process mean y_0 may be unavailable. The

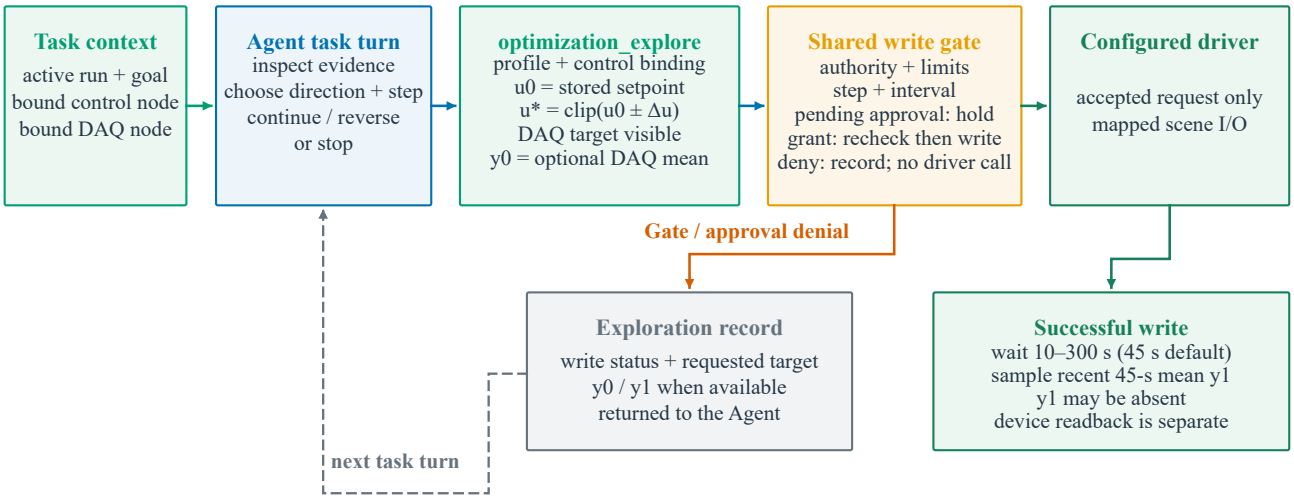


Fig. 3. One task-level call proposes one bounded setpoint step. Profile, binding, DAQ visibility, write limits, and any required approval are checked before driver dispatch. Write-governance or approval denial is recorded without a driver call; preflight errors return before an exploration record. After success, the record returns optional DAQ means separately from device readback. The dashed arrow marks a later Agent task turn, not autonomous continuous control or guaranteed convergence.

write controller resolves current authority and limits, applies configured step and timing guards, and requests approval when required by the route. After approval, it re-evaluates the applicable authorization and write checks before dispatch. A failed profile, control-binding, or target-DAQ precheck returns an error before dispatch and is not persisted as an exploration record. A write-controller denial or approval decline is returned and recorded as a rejected exploration outcome, with no driver call. A driver-stage error can also be stored with status `rejected`, so that status alone does not establish whether dispatch occurred. Once a driver call begins, device state may be uncertain; the caller must read and reconcile it before retrying.

After a successful operation, the service waits for the configured settling interval and attempts to retrieve a recent DAQ mean y_1; this observation may be absent. The implementation queries a trailing 45-s DAQ window after a configurable 10–300-s wait (45 s by default). If the wait is shorter than the window, the result may include pre-write samples. The stored control target is the requested value, not a measured actuator value. The receipt, any driver readback, and the DAQ mean remain separate fields. Fig. 3 shows how the Agent uses the returned record to decide whether to make another call.

This feedback is discrete and task-mediated: the Agent selects each later direction, step, and stopping decision on a subsequent task turn. The runtime does not autonomously iterate to convergence.

V. Operation Governance and Evidence

Section III defines the shared dispatch contract. This section focuses on the checks that follow it: scene-specific write admission, conditional approval, and the records returned by each route.

A. Scene-Specific Write Admission

After the shared dispatch checks pass, the selected handler applies the limits configured for the target scene and control. Authorization and value admission are separate checks. The target band describes a desired process outcome; it does not define the admissible write interval. A candidate may satisfy all write constraints while the measured process value remains outside the target band.

For an online node write, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{L}_n^{\text{base}} \cap \mathcal{L}_n^{\text{prod}}(t) \cap \mathcal{L}_n^{\text{rec}}(t), \qquad (2)$$

where $\mathcal{H}_n$ is the hard engineering range, and $\mathcal{L}_n^{\text{base}}$, $\mathcal{L}_n^{\text{prod}}$, and $\mathcal{L}_n^{\text{rec}}$ are the baseline, active-product, and active-recipe limits. These numerical bounds are distinct from the scope, policy, grant, and binding predicates in Eq. (1). Missing optional limits impose no restriction. An empty intersection rejects the request and identifies the excluding layer. Recipe dispatch checks a replacement recipe against the hard, baseline, and product limits rather than the previous recipe's window. Step, interval, and hold-window limits apply when configured. These application-level checks complement, but do not replace, equipment interlocks.

B. Approval, Execution, and Evidence

Approval behavior depends on the operation route. For a single-node `optimization_explore` request in manual mode, the handler sends the node, requested value, current effective interval, and stated rationale to the approval service, then waits before calling the governed write operation. A decline or timeout returns without reaching the controller write. While the request is pending, the handler does not dispatch. After approval, it rechecks the applicable node binding and channel grant. The shared write controller then evaluates the current engineering, parameter, product, and recipe limits, along with the step and write-interval rules, before dispatching to the configured driver. This single-node path does not use the recipe-operation context snapshot described below.

Recipe operations use a separate approval path. For `recipe_trial` and `recipe_apply`, the approval is

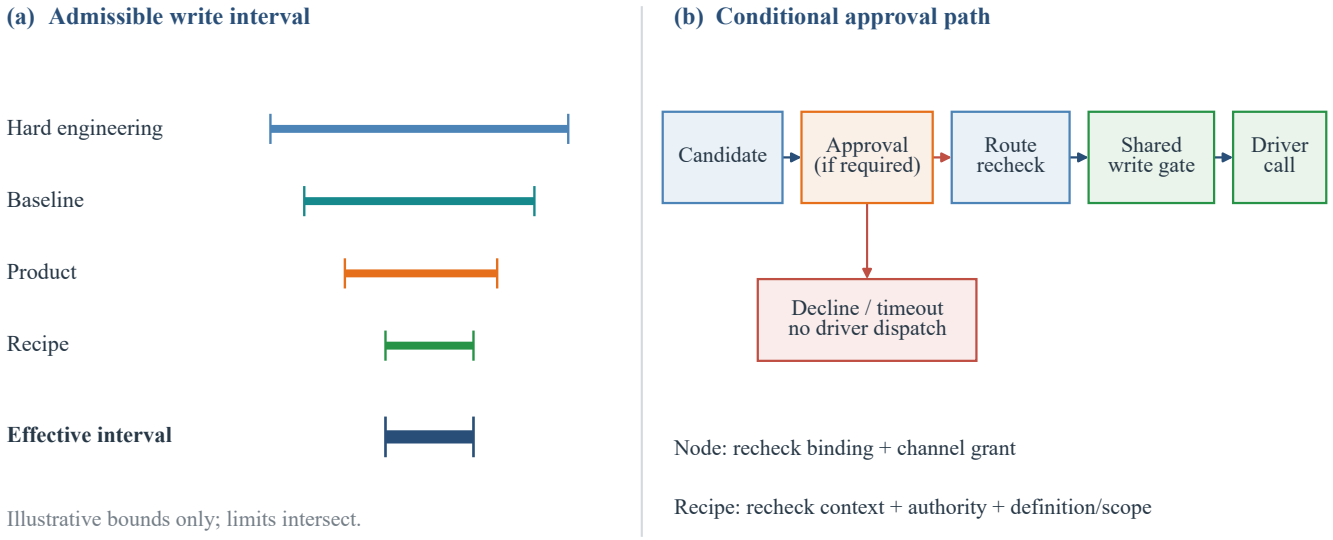


Fig. 4. Configured limits define the admissible write interval, while approval controls dispatch. (a) Conceptual intersection of hard engineering, baseline, product, and recipe limits; horizontal positions are illustrative, not experimental values. (b) Pending requests are held, and decline or timeout prevents dispatch. Approved or non-gated requests pass route-specific rechecks before the shared write controller checks current limits, step, and write interval; node requests recheck the binding and channel grant, whereas recipe requests also recheck context, authority, definition, and scope.

bound to the recipe operation and active line–product–recipe–run context. After approval, the handler checks the authority and context again and verifies the recipe definition and candidate scope before dispatch. A changed context, revoked binding, or changed recipe definition prevents that proposal from proceeding. Recipe trial approvals use a 30-minute window by default; single-node approval waits expire after 180 s by default. Online agent and manual node writes are separated by at least 60 s outside benchmark mode, while recipe trials use a per-line cadence of 300 s by default.

Fig. 4(a) shows the configured bound intersection in Eq. (2); its horizontal positions are schematic and are not run-derived values. Panel (b) abstracts the conditional approval branch. The post-approval checks are route-specific: the single-node path rechecks its binding, channel grant, and current write guards, whereas recipe operations also revalidate the active context and recipe state.

A `recipe_trial` illustrates the recipe-operation path. The handler checks the bound recipe, active run, candidate parameters, and route timing. If approval is required, it requests approval and then revalidates authority and context before submitting a bounded batch through the configured driver. It returns a receipt for each parameter. A trial does not save or apply a recipe version; the caller can retrieve later DAQ observations separately before updating, applying, or rolling back the recipe. The driver registry maps semantic controls to configured protocol drivers, so operation arguments do not contain protocol-specific details.

Receipts record what happened to a command; observations record what followed in the process. Command results are graded as readback-verified, transport-acknowledged, or unverified. A readback mismatch can trigger up to three reads at 2 s intervals, and recipe batches retain verification outcomes for each parameter. A readback-verified result means the returned value matched the request. A transport acknowledgement alone does not establish that the device applied the value, and an unverified result does not establish a matching readback. DAQ stores later timestamped process observations, while recipe history, approvals, and task records remain with their owning services. Identifiers and timestamps can link these records when available, but a later observation alone does not prove that the command caused the measured change. Section VI evaluates selected parts of this evidence chain under the stated test conditions.

VI. Experimental Evaluation

The evaluation covers three parts of the runtime path: whether selected scene–protocol mappings accept admissible writes, reject collected structural-range requests, and return matching readbacks where available; whether the governed-write path supports deterministic simulator missions; and whether the process-window check blocks selected excursions before they reach a mock driver. Approval and authorization receive separate route checks. Because the blocks use different units and fixtures, we report their counts separately rather than pooling pipeline assertions, node probes, seed outcomes, and guard injections.

A. Test Conditions and Counting Units

Two integrated pipeline executions, documented in Supplementary File S7, each reported 77 passing assertions, zero warnings, and zero failed assertions. The runner reused a platform and simulator that were already running. The manifests identify commit `da627d21` and record `agentHarness=(none)`; these executions therefore exercised configured service paths without an active language model or live harness. The 77 checks are assertions within each

TABLE III
EXPERIMENTAL BLOCKS, COUNTING UNITS, AND SUPPORTED CLAIMS. RESULTS REMAIN SEPARATED BY BLOCK BECAUSE THEIR DENOMINATORS AND TEST FIXTURES DIFFER.

Block	Intervention and unit	Recorded outcome	Evidence boundary
Scene–protocol probes	Configured node requests across four scenes; admissible values include three interior points and the tested upper boundary, with structural-range probes in a separate set.	220/220 admissible requests accepted; 219/219 collected structural probes rejected; 216/216 available readbacks matched.	Selected mappings only; the planned scene–protocol matrix is partial.
Policy comparison	Five matched simulator seeds across three scenes. C0 held for 60 s without writes; C1 ran a deterministic scenario mission through the governed-write service.	The designated process variable ended in band for 0/15 C0 and 15/15 C1 endpoints.	Simulator evidence; no live Agent or competing controller. Runner mission flags are not compared across conditions.
Process-window ablation	Four guard configurations, three repetitions each; per repetition: six negative, three nominal legal, and three boundary requests.	0/6 targeted window violations reached the mock driver with the interlock; 6/6 reached it without the interlock. Legal false blocks were 0/9 in each arm.	Mock driver and benchmark mode; no readback-mismatch stimulus.
Route checks	Separate approval pilot (four calls), production-mode guards (one example per named guard), and authorization probe (three calls).	Expected approval, rejection, range, and authorization branches were observed for the configured examples.	Path demonstrations only; no human decision study or rate estimate.

execution, not 77 independent trials. Supplementary File S7 contains the run records and archived source summaries.

The scene–protocol probe records each requested value, expected decision, actual outcome, and available readback; its unit is one configured node request. For the optimization comparison, the unit is a seed–scene pair. C0 held each simulator for 60 s without writes, whereas C1 used a deterministic scripted policy through the governed-write path. Each C1 seed ran for 150.41–172.90 s. Because exposure time and mission logic differed, the comparison describes the tested conditions but does not isolate the effect of the write policy. The process-window ablation used three repetitions per arm. Each repetition included six negative probes, three nominal legal requests, and three boundary requests. The ablation ran in benchmark mode against a mock driver, so its outcomes assess the software interlock in that fixture rather than plant-level protection. Supplementary File S7 preserves the source CSVs and per-run records.

B. Configured Scene and Protocol Paths

The valid raw probe run covers 55 configured writable nodes across four simulator scenes and five protocol families: Modbus over Transmission Control Protocol (TCP), Modbus remote terminal unit (RTU), OPC UA, MQTT, and Hypertext Transfer Protocol (HTTP). Each scene used four of the five families. Table IV reports the node allocation by scene and separates accepted in-range and upper-boundary requests from structural negative probes. These counts describe configured mappings, not throughput or complete scene–protocol matrix coverage. Readback consistency is reported only for accepted requests on routes that returned an actuator value. One biaxial-line structural case was not collected, and HTTP routes did not expose actuator readback. The intended complete scene–protocol matrix remains partial; these results describe the tested mappings rather than universal protocol or scene portability.

For each writable node, the expected-acceptance set comprised three interior values and the tested upper boundary; the structural set comprised four values outside the configured engineering range. Across the collected subset, all 220 admissible requests were accepted, all 219 collected structural probes were rejected, and all 216 available actuator readbacks matched their requests. These counts characterize the configured examples; they do not estimate false-block rates or establish complete matrix coverage. In a separate configuration check, the integrated pipeline loaded a film-line preset with five endpoint types and rejected nine of nine selected negative probes without code changes. Because the runtime was reused, this was not an independent cold-start deployment.

C. Target-Band Comparison

The same five seeds, 42–46, were used in the injection, wastewater, and annealing scenes. Both conditions began from their scenario blueprint and seeded simulator state. C0 made no writes for 60 s; its endpoint was the mean of the final five process samples. C1 ran a deterministic scenario mission through the shared governed-write service, queried DAQ after configured settling intervals, and applied scenario-specific guard or recovery logic. The policy was scripted, not Agent-generated. C1 took 150.41–172.90 s per seed, so the conditions differed in exposure time and mission logic.

The comparison asks whether each recorded terminal process-variable statistic fell within its configured target band. For C0, that statistic is the mean of the final five samples; for C1, it is the mission-terminal reading. The runner's `attained` field is not comparable across conditions because C0 combines the process-variable band with configured guards, whereas C1 uses scenario-specific mission criteria. We therefore recomputed band membership from the raw `pv`, `target`, and `tol` columns.

For injection mass, the target was 32.5 ± 0.35 g. C0's terminal means were 31.206–31.272 g and C1's mission-terminal readings were 32.173–32.340 g, placing 0/5 and 5/5 recorded statistics, respectively, inside the band. For wastewater dissolved oxygen (DO), the target was 3.4 ± 0.6 mg/L; C0 terminal means were 0.198–0.268 mg/L and C1 readings were 3.576–3.864 mg/L, again giving 0/5 versus 5/5 in-band statistics. In annealing, the target was 95 ± 6 on the Vickers hardness scale (HV); C0 terminal means were 134.36–134.90 HV and C1 readings were 97.44–99.90 HV.

TABLE IV
OBSERVED SCENE–PROTOCOL PROBE OUTCOMES. THE TABLE REPORTS THE COLLECTED SUBSET; READBACK DENOMINATORS INCLUDE ONLY ACCEPTED REQUESTS WITH AN AVAILABLE ACTUATOR VALUE.

Scene	Writable nodes	Writable nodes by protocol	Admissible accepted	Structural rejected	Readback matched
Injection	11	TCP 5; RTU 2; OPC UA 3; MQTT 1	44/44	44/44	44/44
Wastewater	7	TCP 1; RTU 3; OPC UA 2; MQTT 1	28/28	28/28	28/28
Annealing	7	TCP 3; RTU 2; OPC UA 1; MQTT 1	28/28	28/28	28/28
Biaxial line	30	TCP 20; RTU 1; OPC UA 8; HTTP 1	120/120	119/119	116/116

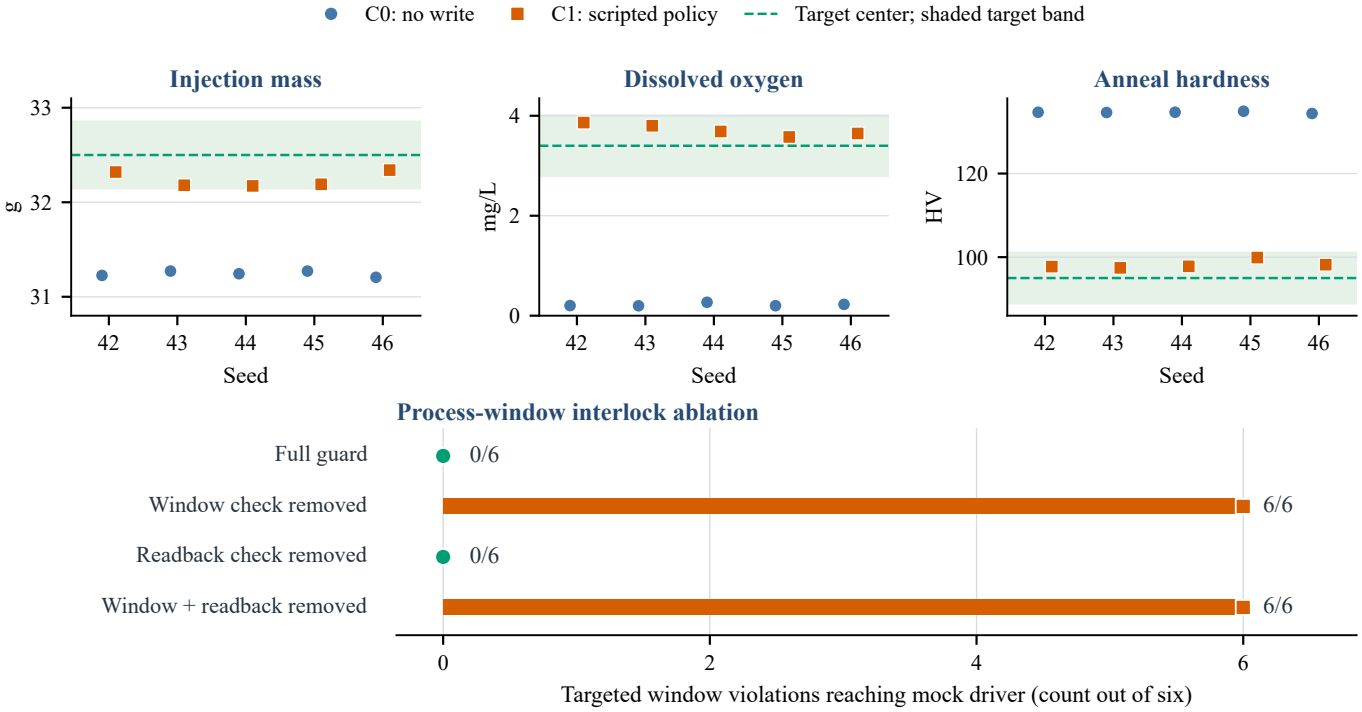


Fig. 5. The scripted policy reached its configured target band in all 15 seed–scene endpoints, whereas none of the 15 no-write endpoints did; removing the process-window interlock sent all six targeted violations to the mock driver. Top: individual terminal process-variable statistics for seeds 42–46 in the injection, wastewater dissolved-oxygen, and annealing scenes. C0 is the 60-s no-write condition and C1 the deterministic scripted policy; each C0 value is the mean of the final five samples, whereas each C1 value is the mission-terminal reading. Shaded regions and dashed lines mark the target bands and their centers. Bottom: targeted process-window violations reaching the mock driver under each guard configuration (six probes per arm, three repetitions). The window check is removed in the second and fourth arms; the readback check is removed in the third and fourth arms. Structural checks remain active in all arms.

Across the 15 designated process-variable outcomes, all 15 C1 readings and none of the 15 C0 terminal means were within their respective bands. Across the five C1 seeds, the scripted missions issued 12 writes in 17 rounds for injection, 56 in 59 for wastewater, and 41 in 50 for annealing. These counts describe workload, not efficiency. Because the endpoint conditions were not time-matched and used different mission logic, the contrast is descriptive and does not compare the runtime with an established controller. Fig. 5 shows each endpoint and its configured band.

D. Process-Window Interlock Ablation

The primary contrast was the full guard against the same configuration with only the process-window interlock removed. With the guard enabled, 18/18 negative probes were rejected before the mock driver and 0/6 targeted window violations reached it. Removing that interlock reduced the negative-probe rejections to 12/18: all six targeted window violations reached the driver, while the 12 out-of-range structural probes remained blocked. Each arm also accepted 9/9 nominal legal requests without a false block and met its configured boundary expectations in 9/9 checks.

The no-readback arm retained the process-window interlock and produced the same 18/18 rejection result. The combined no-window/no-readback arm matched the no-interlock arm on these stimuli. No readback-mismatch stimulus was included, so the experiment isolates the observed process-window effect but says nothing about readback-check effectiveness or interaction between the two checks. The implementation has separate production-mode tests for the interval, step, node-range, recipe-window, and hold-window guards; each configured case returned its expected rejection code, while the legal baseline was accepted. Those single-case checks are route demonstrations, not rate estimates.

E. Approval and Authorization Path Checks

A separate four-call simulator pilot exercised the approval route. In manual mode, the scripted approval case changed the control value from 202 to 204.8, while the scripted rejection left it at 216. The auto-mode calls also executed, but their

starting values do not match the manual cases named in the runner's pairing labels. Because the decisions came from a test fixture, no operator made the decisions, and the requests do not form a clean matched approval-on/off comparison, we treat these records as a path check rather than an approval-effect estimate. The full values and decision trace are retained in S7.

A separate authorization check rejected one unbound write and one cross-node write, while accepting one write to a bound control node. Together, these examples showed that the configured checks returned their expected outcomes in the tested simulator routes. They do not establish a general safety rate, physical-equipment behavior, or live-Agent decision quality.

VII. Discussion

The scene–protocol probes and simulator missions address different parts of the integration claim. The probes show that the selected configurations resolved to the intended node and driver paths. The missions show that a deterministic policy issued governed writes and returned process endpoints in the tested simulator scenes. The target-band contrast does not isolate the effect of the write path or establish control performance: the no-write and scripted conditions differed in duration, mission logic, and endpoint summaries. The planned scene–protocol matrix remains incomplete, and the film-line configuration check reused a running platform rather than testing a cold deployment.

The process-window ablation supports a narrower causal conclusion. Removing the check allowed all six targeted excursions to reach the mock driver while structural-range checks remained active. This isolates the process-window predicate in the tested fixture. The ablation ran in benchmark mode and included no readback-mismatch stimulus, so it does not assess the full production guard stack or readback-check effectiveness. Separate production-mode examples exercised the interval, step, node-range, recipe-window, and hold-window branches. These are route checks, not estimates of rejection rates under load.

The approval and harness claims are also bounded by the tests. The approval pilot used scripted decisions, included no operator interaction, and lacked matched approval-on/off conditions; it therefore cannot establish a human-in-the-loop interception effect. The runtime exposes a common operation contract to registered harness adapters, but no active Agent harness was used in these runs. Harness replacement and agent decision quality remain untested.

The evidence model also constrains how future deployments should be evaluated. A command receipt records the write outcome; an actuator readback reports the returned control value when available; and a DAQ sample records a later process measurement. The optimization path checks that its target DAQ binding is visible, but these tests do not verify that every configured observation is the commissioned response to the selected control point. The DAQ query uses a trailing 45-s window, so a shorter settling delay may leave pre-write samples in the result. Before using these measurements to assess a write, each scene needs a verified control–measurement pairing and observation delay.

The integrated-pipeline manifests identify the reported commit and show that the platform and simulator were already running. They do not capture a hermetic snapshot of all active service source. Supplementary File S7 retains the runner inputs and records needed to audit the reported runs. Application-level checks supplement PLC, SIS, and equipment protection; the simulator and mock-driver tests do not establish physical-plant safety.

VIII. Conclusion

This work presents a configuration-driven runtime that binds production context and semantic signals to named operations, registered protocol drivers, and shared write governance. Across the collected four-scene simulator probes, 220/220 admissible requests were accepted, 219/219 collected structural-range requests were rejected, and 216/216 available actuator readbacks matched. In the three-scene mission comparison, 15/15 scripted-policy endpoints fell within their target bands, compared with 0/15 no-write endpoints; this descriptive contrast used different exposure times and mission logic. In the mock-driver ablation, the process-window check blocked all six targeted violations, whereas removing it forwarded all six while structural checks remained active. In the tested simulation and mock-driver settings, the results demonstrate the selected scene mappings, scripted-write path, and process-window guard. Live-Agent and harness behavior, human approval effects, and physical-plant protection remain unevaluated.

Acknowledgment

The authors used artificial-intelligence tools for language editing and take responsibility for the manuscript. Funding and author identities are withheld in this review copy.

Author Contributions

Individual CRediT contributions are declared in the submission system and withheld in this review copy.

Declaration of Competing Interest

The authors declare no known competing financial interests or personal relationships that could have appeared to influence this work.

Data Availability

Supplementary File S7 contains the selected current-run manifests and CSVs, the per-seed optimization outcomes, ablation records, and a self-contained visual report. The reported commit and harness hashes are provenance fields rather than runtime signatures; the package contains no physical-plant data.

REFERENCES

[1] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[2] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[3] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[4] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. IEEE 64th Conf. on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[10] Y. Liu, X. Xu, and J. Polzer, "Agentic AI-enabled semantic commissioning of a cognitive digital twin for reconfigurable manufacturing," in *Proc. IEEE 22nd Int. Conf. on Automation Science and Engineering (CASE)*, 2026, doi: 10.1109/CASE69030.2026.11704436.

[11] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[12] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *Proc. IEEE 8th Int. Conf. on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[13] M. Xavier, M. Jolly, V. M A, and M. Xavier, "IndustriConnect: MCP adapters and mock-first evaluation for AI-assisted industrial operations," 2026, arXiv:2603.24703 [cs.SE]. [Online]. Available: https://arxiv.org/abs/2603.24703

[14] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[15] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[16] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[17] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[18] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[19] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[20] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[21] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[22] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.